

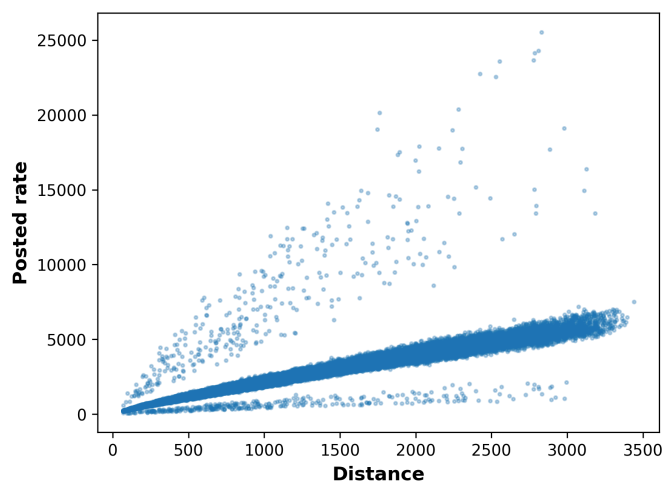
Freight rate prediction - Spotter

The goal is to predict freight rates depending on various factors such as the route distance, market rates, weight of the shipment and date ..etc

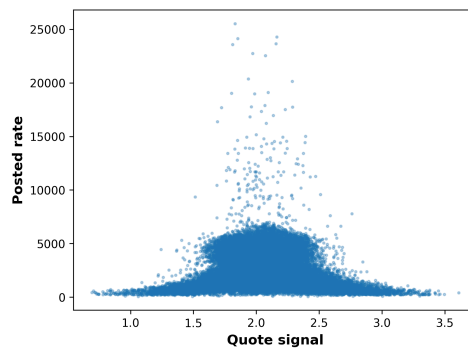
- Code link: <https://github.com/Rachelslh/freight-rate-prediction.git>
- Loom video: <https://www.loom.com/share/8f08f9285ae248b3968b6630ad2d31c4>

Data exploration

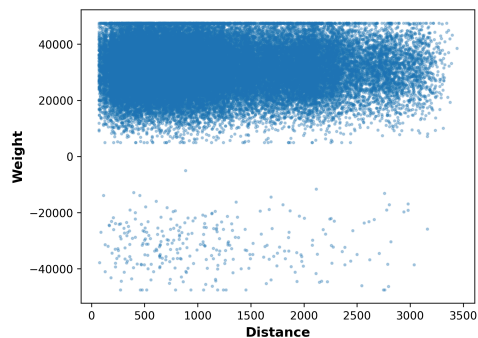
- The supplied dataset contains 12 features, where each feature might have some correlation with the target that is the **ground-truth rate**
- Decided to investigate the type of relationship across features here, to find out if it's linear or not;
 - Posted rate & distance: Most important one is whether the posted rate correlates with the distance (as it should) ⇒ The plot clearly shows a linear correlation where the rate increases with the distance although there are some outliers in the dataset



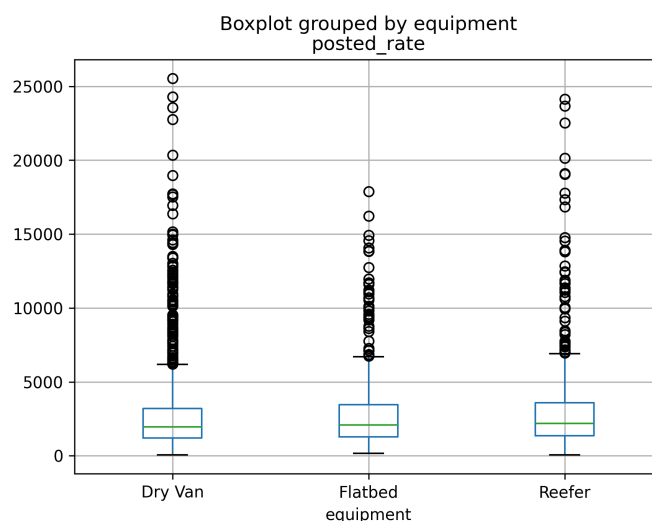
- Posted rate & quote signal: This one looks like a gaussian, distributed around some mean value, this is not a linear relationship



- Distance & weight: not a linear relationship

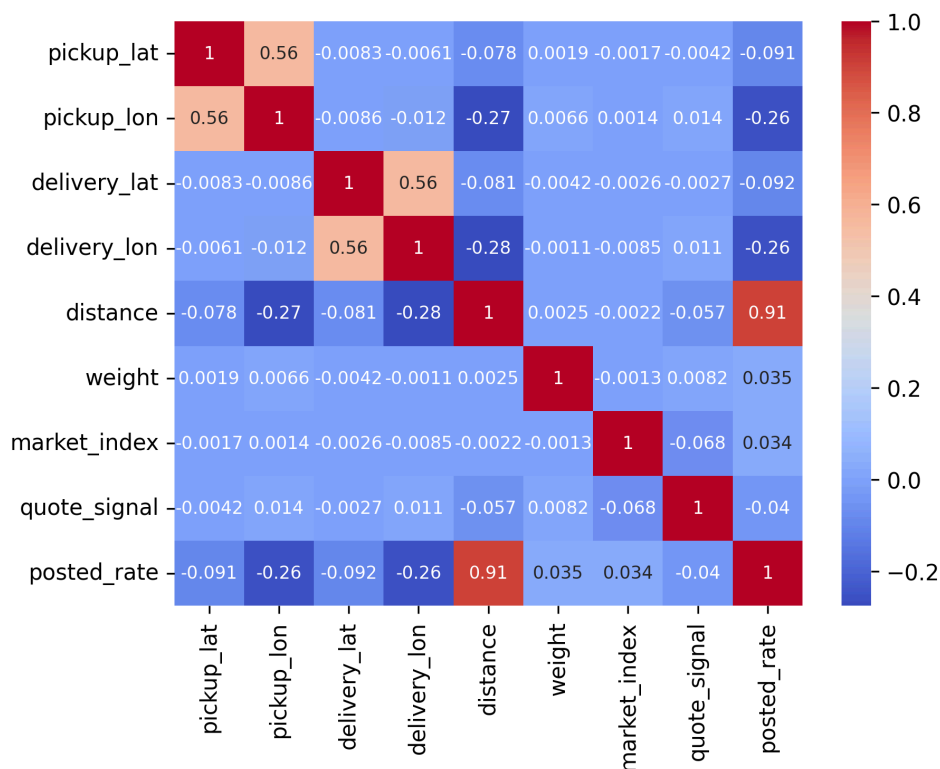


- Posted rate & equipment: Equipment type shifts the typical rate somewhat (Reefer > Flatbed > Dry Van), but it doesn't explain the extreme high-rate outliers — those show up across all three types roughly equally, meaning whatever drives those extreme loads (rush timing, unusual routes, etc.) is independent of equipment type.



- Also plotted a heatmap that shows pearson-based(linear) correlations across the all numerical raw features that we have, also tried Kendall and

Spearman for non-linear relationships but that did not give any promising results



The heatmap shows that the rate is essentially affected by the distance here, but this does not mean we cannot engineer more complex features that may affect the final outcome.

- The dataset contains rows where some weight and market index values are missing, i used the median values to pre-fill missing ones in order to avoid throwing away meaningful information and to mimic real-world datasets where sometimes the information gets lost

Feature engineering

Some interesting features that can be formed out of the raw ones we have here are:

- Interaction-based features: Purely multiplicative
 - distance * quote signal
 - distance * marked index
 - distance * weight

- Date-related feature:
 - Month of the date, some months can be busier than others depending on seasonal shifts
 - Holidays $\Rightarrow$ Rates should increase on holidays [I did not add this feature in the code]
- Distance features:
 - Haversine distance $\Rightarrow$ Given that we have latitude and longitude parameters, the Haversine formula calculates the shortest distance between two points on the surface of a sphere, which is very accurate compared to flat distance such as Euclidean and Manhattan distances.
- Missing feature values:
 - weight missing: Some records in the data have null weight values, this is reflective of what real-world data can be like and should be accounted for.
 - market index missing: Same as above
- Frequency features:
 - Route frequency: Answers the question of how many times is the exact same route done?
 - Pickup frequency per city
 - Delivery frequency per city

Data split

- The data is a time-series type of data, meaning the split should be based on date; This means the training split uses past data and the test split uses future data $\Rightarrow$ This is a way for us to mimic production pipelines.
- 75% of the data is allocated to training, the remaining 15% is for testing our model

Modeling

- Initially, i tried using a simple linear regression where the rate is a linear function of the distance alone because of how much linear correlation there is between the 2 $\Rightarrow$ This turned out to be a baseline model compared to my second approach (which you will find in the code).

- Since we have so many raw and engineered features, decided to use Gradient boosting combined with squared error as the objective function.
 - Decision trees use many parameters and finding the right values can be time-consuming, this is why i used Optuna to optimize and find the best parameters to run gradient boosting. A json file containing best params is automatically saved.
- I have also applied a logarithmic transformation on the rate target to stabilize the right-skewed rate values (there are some outliers in the data that have very large rates).

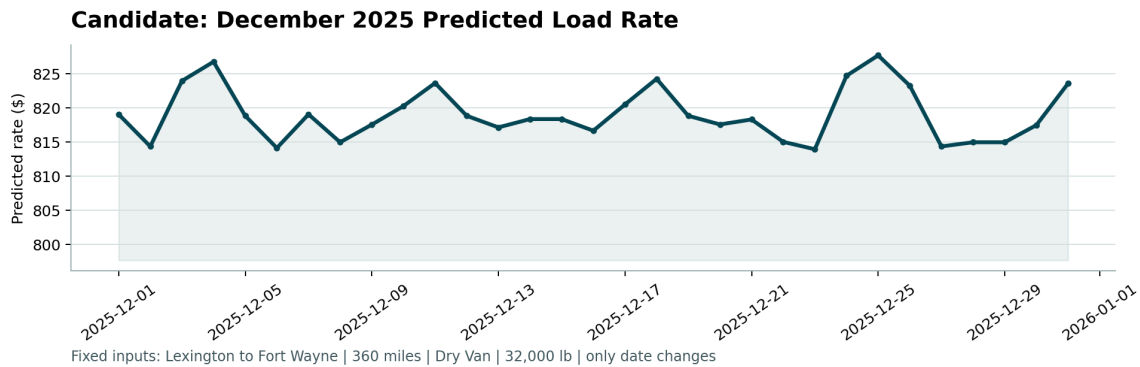
Training

- Trained for 1017 iterations, early stopping configured after 50 iterations with non-improved loss.
- Most important features that seem to affect the rate prediction are: (This is showing feature importances/coefficients from the model itself post-training)

Feature	Importance
distance	0.352572
haversine_dist	0.307628
distance_x_quote	0.092010
distance_x_weight	0.086827

Evaluation - Validation

- Essentially used MAPE (mean average percentage error) metric to measure how is the model generalizing to the test split. With fine-tuned parameters, it got it down to 5.08% of error.
- Also used R-squared, to measure how close my predictions are to the actual values, this reached 84.12%
- Generated December chart



- Some caveats worth mentioning here: The CSV file for December inputs is clearly missing many features such as latitude, longitude, market index and quote signal, i had to derive these features from the validation set in order to have the model predict something meaningful. Market index and quote signal were created as mean values of the December data in the validation set.
- Predictions clustered in the ~814–828 range, with a repeating oscillation driven by daily shifts in the aggregated market signal
- Scorer output

```

(base) MacBook-Pro-de-racha:Spotter racha$ uv run score.py --predictions validation_predictions.csv --december-predictions data/december-chart-inputs.csv
Validated 12,000 final predictions.
Validated 31 fixed December predictions.
Created chart: scorer_results/candidate_december.png
Final validation metrics are calculated by Spotter after submission.

```

Tech stack

- Python
- UV ⇒ Simplifies running scripts and automates dependencies installation
- Xgboost ⇒ Gradient boosting library
- Scikit-learn ⇒ to use predefined error metrics such as MAPE
- Omegaconf ⇒ I used a yaml config to parametrize the entire pipeline
- Optuna ⇒ To optimize for best parameters
- Numpy/Pandas
- Pickle ⇒ For artifacts